Evolution of an Internal Reward Function for Reinforcement Learning

Weiyi Zuo
weiyi_zuo@foxmail.com
University of Chinese Academy of Sciences
Beijing, China

Joachim Winther Pedersen
jwin@itu.dk
IT University of Copenhagen
Copenhagen, Denmark

Sebastian Risi
sebr@itu.dk
IT University of Copenhagen
Copenhagen, Denmark

ABSTRACT

Artificial neural networks (ANNs) can be trained with reinforcement learning (RL) in simulation to control robots. However, changes in the environment resulting in out-of-distribution situations put learned policies at risk of failure. Since the world can change in unpredictable ways, it might be desirable to be able to continue to update the parameters of the ANNs even after deployment, to prevent failures stemming from a distributional shift. However, in order to optimize with RL, a reward signal is needed. This is usually provided in the simulated environment, but might not necessarily always be available after training. We propose a solution to this problem that involves evolving a function that provides a reward signal to an RL algorithm based only on the inputs and outputs of the policy. We call this approach Evolved Internal Reward Reinforcement Learning (EIR-RL) and test it on various control tasks that have different reward structures and difficulty levels. Our method shows improved training stability and speed of the RL agent under standard circumstances, as well as the ability to train the RL agent under circumstances unseen during the initial optimization phase. We discuss how our results could inform future studies on autonomous, adapting agents.

ACM Reference Format:
Weiyi Zuo, Joachim Winther Pedersen, and Sebastian Risi. 2023. Evolution of an Internal Reward Function for Reinforcement Learning. In *Genetic and Evolutionary Computation Conference Companion (GECCO '23 Companion), July 15–19, 2023, Lisbon, Portugal.* ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/3583133.3590610

1 INTRODUCTION

The past decade has seen much progress in the research fields of both deep learning and robotics. However, in dynamic environments, there is a risk that environmental circumstances change over time and in unpredictable ways. Policies learned with reinforcement learning are notoriously brittle, meaning that even small changes in the environment can lead to failures [7]. Unfortunately, the reward signal that was used during simulation might not be available to use after training, e.g., in the real world Smith et al.. For this reason, it would be useful for the agent to be able to estimate a reward signal that it can learn from, using only information that is always available to it.

With this goal in mind, we introduce Evolved Internal Reward Reinforcement Learning (EIR-RL). As the name suggests, we evolve a function that creates a signal that can be used to optimize an RL agent. As long as the original goal stays the same, the agent can optimize its parameters indefinitely, and recover performance even in the face of a distributional shift. The evolved function can do this without relying on information beyond the inputs and outputs of the agent. We test the EIR-RL approach on various reinforcement learning tasks of differing difficulty and reward structures. After optimization, EIR-RL is found to produce enhanced training speed and training stability of the RL agent, as well as the ability to recover performance in a changing environment without the need to refer to a reward signal granted by the environment.

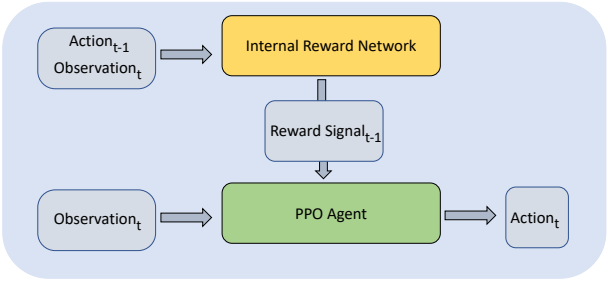

Figure 1: Overview of EIR-RL. The Internal Reward Network takes inputs that are readily available to the PPO agent and outputs a reward signal that is used to update the parameters of the networks of the PPO.

2 APPROACH: EVOLVING INTERNAL REWARD FOR REINFORCEMENT LEARNING

The goal of EIR-RL is to obtain a function that can produce a reward signal that can be used for RL using only information that is directly available for the RL agent. We achieve this by optimizing the parameters of a neural network. We call this the Internal Reward (IR) network.

As input, the IR network takes the same observation as the policy network of the RL agent, as well as the resulting output of the policy network. The IR network outputs a single scalar that is used for optimizing the policy using Proximate Policy Optimization (PPO) [5] instead of the reward scalar provided by the environment. Our approach thus requires two optimization processes: an inner- and

GECCO '23 Companion, July 15–19, 2023, Lisbon, Portugal

ACM ISBN 979-8-4007-0120-7/23/07.
https://doi.org/10.1145/3583133.3590610

an outer loop, as described in Algorithm 1, where 'ES' denotes evolution strategy, 'IRN' denotes internal reward network, 'ENV' denotes environment, 'ir' and 'er' denotes internal and extrinsic reward, respectively.

We use an evolution strategy (see Section 2.1) for the outer loop optimization of the IR network. In each generation, a population of IR networks is generated. The fitness of each IR individual is determined by evaluating the score of a policy that was optimized using the IR output in place of the reward provided by the environment. Importantly, the fitness is based on the true score in the environment, not the evolved reward signal. The parameters of the IR network are updated through evolution, and stay fixed during the PPO training phase. From the point of view of evolution, each individual is born with a reward system in form of the IR network and a learning system in the form of the PPO. Together, these innate systems provide a way of shaping a random policy into a functioning one. The fittest individuals will be the ones that have the reward system most suited for learning under the constraints of an objective function determined by the external environment.

Algorithm 1: Meta-training of EIR-RL

```
Initialize ES;
[Outer Loop] for epoch = 1,··· do
    [Inner Loop] for agent = 1, ··· , population_size do
        Sample vector ε from ES;
        Initialize IRN parameters with ε;
        Initialize POLICY network parameter φ randomly;
        Sample ENV from task distribution;
        Initialize state from ENV;
        for t = 1,··· , max_t do
            action = POLICY(state) ;          # Agent part
            ir = IRN(action, state);
            POLICY.update(ir) ;           # Based on Eq.1
            er, state = ENV(action) ;  # Environment part
        end
        fitness = SUM(er) for each t;
    end
    Update ES by the fitness of each agent;
end
```

2.1 Optimization

For the outer loop optimization of the IR network parameters, we use a variant of the family of evolution strategies called Co-variance Matrix Adaptation Evolution Strategy (CMA-ES) [2]. Evolution strategies do not require the calculation of explicit gradients and has the ability to learn over long time horizons. A particular advantage of CMA-ES is the reduced reliance on a large population size in order to work well. Calculating the co-variance matrix of a set of parameters can be expensive when the number of parameters is large (on the order of 10000). However, in our experiments the IR networks are all small-sized neural networks with a relatively low number of parameters, making CMA-ES a favorable gradient-free candidate for the outer loop optimization of EIR-RL.

The fitness of an IR network is the performance of a policy that has been optimized using the IR signals as reward. As mentioned above, the RL algorithm used for maximizing the IR signal was Proximal Policy Optimization (PPO) [5]. Different from the general PPO, the output of the internal reward network replaces the original extrinsic reward at each time-step to train PPO (as shown in Fig. 1). In this case, the loss gradient of the policy is expressed as:

$$\nabla_\theta J(\theta) \sim \sum_{t=0}^{T} \nabla_\theta \log \pi_\theta(a_t|s_t)(R_\varepsilon(a_{t+1}, s_{t+1}) + \gamma V_v(s_{t+1}) - V_v(s_t)) \tag{1}$$

In this loss, there are three functions parameterized by neural networks: internal reward network R_ε, actor network π_θ and value network V_v. The three networks assume orthogonal functions. The actor network outputs the actions of policy; the value network outputs the expected return under current strategy; the internal reward network outputs estimated policy-independent rewards from Markov Decision Process. Actor and value networks are updated based on the loss by using the PPO method [5].

3 RESULTS

Here we show the results of evaluating the optimized IR networks in different environments. We evaluate models on default environmental settings that were used for training, as well as altered settings novel to trained IR networks. In the plots below, 'Oracle' refers to a PPO trained with the reward provided by the environment. This is to emphasize that the primary motivation of EIR-RL is to enable optimization in the absence of externally provided reward signals. PPOs trained with either reward signal have the same specifications in terms of network sizes and other hyperparameters.

3.1 Lunar Lander

In the `LunarLander-v2` environment, the goal is to safely guide a small rocket ship to land within the correct landing grounds. Due to this mix of continuous and sparse reward signals, this environment presents an interesting challenge for the EIR-RL method.

For the outer loop optimization, a population size of 64 is used, and optimization runs for 500 generations. The internal reward network is an RNN with {9,32,32,32,1} nodes per layer, where the last layer of size 32 is a non-recurrent dense layer to reduce the output to a single scalar, resulting in 3,521 parameters to be optimized by CMA-ES in the outer loop. As input, the IR network took the observation from the environment as well as the discrete action of the policy network from the previous time step. The fitness of an IR network was determined after 100,000 time steps in the environment. The PPO updated policy- and critic parameters every 1024 steps, with a batch size of 32. Both the policy network and the critic network of the PPO had two hidden layers of 64 neurons activated by ReLU.

After the optimization of the IR network on the `LunarLander-v2` environment, we first compared PPO training with the reward given by the environment, and the reward from the IR network. The use of either rewardsignal did result in policies that could score above 200 in the environment. However, only the PPOs trained with the evolved reward signal were able to achieve and sustain a high

Figure 2: Lunar Lander: PPO Training Curves. Means and standard deviations over 5 different PPO runs. Plotted is the moving average of the objective (non-evolved) scores of the latest 20 episodes. The EIR-RL is able to recover most of the performance when strong wind is applied.

moving average over the latest 100 episodes, indicating improved training stability.

In Figure 2, the PPO is first optimized for 400 episodes in the default environmental settings. Then, in the following 800 episodes, a strong wind (wind level 20 is the strongest possible in `LunarLander-v2`) is applied in the environment, making it harder to steer the rocket ship. The performance of EIR-RL initially suffers, but then gradually improves and is consistently well above a score of 100, which indicates that the rocket ship is not crashing. Once again, the PPO trained with the original rewards from the environment suffers from training instability and fails to sustain a high performance. The curve labeled "No Extra Training" shows how an agent trained only in the first 400 episodes performs when the wind is applied. As can be seen, the continued training with the original rewards from the environment results in a training curve similar to simply not optimizing anymore.

3.2 Quadruped

This locomotion environment revolves around training the agent to control a simulated three-dimensional robot with four legs. The goal is to get the robot to walk as far as possible in a straight line. The input to the agent is a vector of 27 elements, containing proprioceptive information about the robot. At each time step, the agent needs to output a vector of eight elements, describing the torque that is to be applied to each of the eight rotors of the robot, respectively.

In the `Ant-v3` environment, we run the training in two different scenarios. The first is simply the default setting of this environment. In the second scenario, one of the legs of the robot is sometimes shortened. Specifically, in each new episode, there is a probability 20% that no change is applied to the leg. The rest of the time, the length of the leg is sampled to be between 0.01 and 0.4 with uniform probability, where 0.4 is the original leg length. Only the third leg of the robot is selected for reduction, whereas all other legs are always maintained at the standard length. Earlier studies have shown that shortening a single leg of a quadruped by approximately 10% can significantly decrease the performance of a policy optimized with standard leg lengths only [4]. In general, increasing the distribution of tasks during meta-training can improve the generalizability of a policy [1, 3], so here we test whether the same is true for an evolved reward function. Apart from the leg-shortening in one scenario, all settings of the training are identical between the two scenarios.

The population for CMA-ES is set to 80, and optimization is run for 1,500 generations. In the inner loop, the PPO agent is allowed 500,000 time steps (at least 200 episodes) in this environment. The PPO is updated every 2048 time steps based on internal rewards The internal reward network is an RNN with {35,32,32,32,1} nodes per layer, where the last layer of size 32 is a non-recurrent dense layer to reduce the output to a single scalar. Thus a total of 4, 353 parameters are optimized by CMA-ES. The PPO consists of the policy feedforward network with {27,32,32,8} nodes per layer and the critic feedforward network with {27, 32, 16, 1} nodes per layer. All hidden neurons are activated by the hyperbolic tangent function.

In this more challenging environment, we pre-train the IR network before optimizing it in the outer loop. We do this by training a single PPO to perform well in the `Ant-v3` environment. Throughout this process, we collect the observations, actions, and external rewards for each time step. With this data set, the pre-training process is simply to optimize the IR network via gradient descent on the supervised regression problem with observations and actions as the input and the reward as the target. Preliminary experiments showed that the pre-trained internal reward network has a similar performance to environmental reward when training a new PPO network. Finally, we used the parameters of the pre-trained internal reward network as the initial parameters of the evolutionary strategy.

Figure 3 shows training curves under different out-of-distribution scenarios. In all cases, the PPO was first trained to perform well on the default task, and the plots show attempts to recover performance when the environmental setting is changed afterward. Three OOD leg length scenarios were examined for each of the four legs of the robot: fixed at 0.35, fixed at 0.01, and gradually shortened from 0.35 to 0.01. The shown curves are the averaged curves over 10 different PPO runs.

As can be seen, the EIR-RL trained with different sampled leg lengths (blue curves) of the third leg performs better when the third leg is changed during testing. The EIR-RL trained with sampled leg lengths has better performance only in one other scenario, that being when the fourth leg of quadruped robot is gradually shortened from 0.35 to 0.01. However, the three methods have similar performance overall. Notably, no disadvantage in terms of training speed or performance can be detected in any of the scenarios when using the EIR-RL trained with sampled leg lengths.

In short, for new scenarios within the meta-training distribution, the training speed of EIR-RL is faster than that of 'Oracle'. In the out-of-distribution scenarios, the training speed of EIR-RL and 'Oracle' are similar.

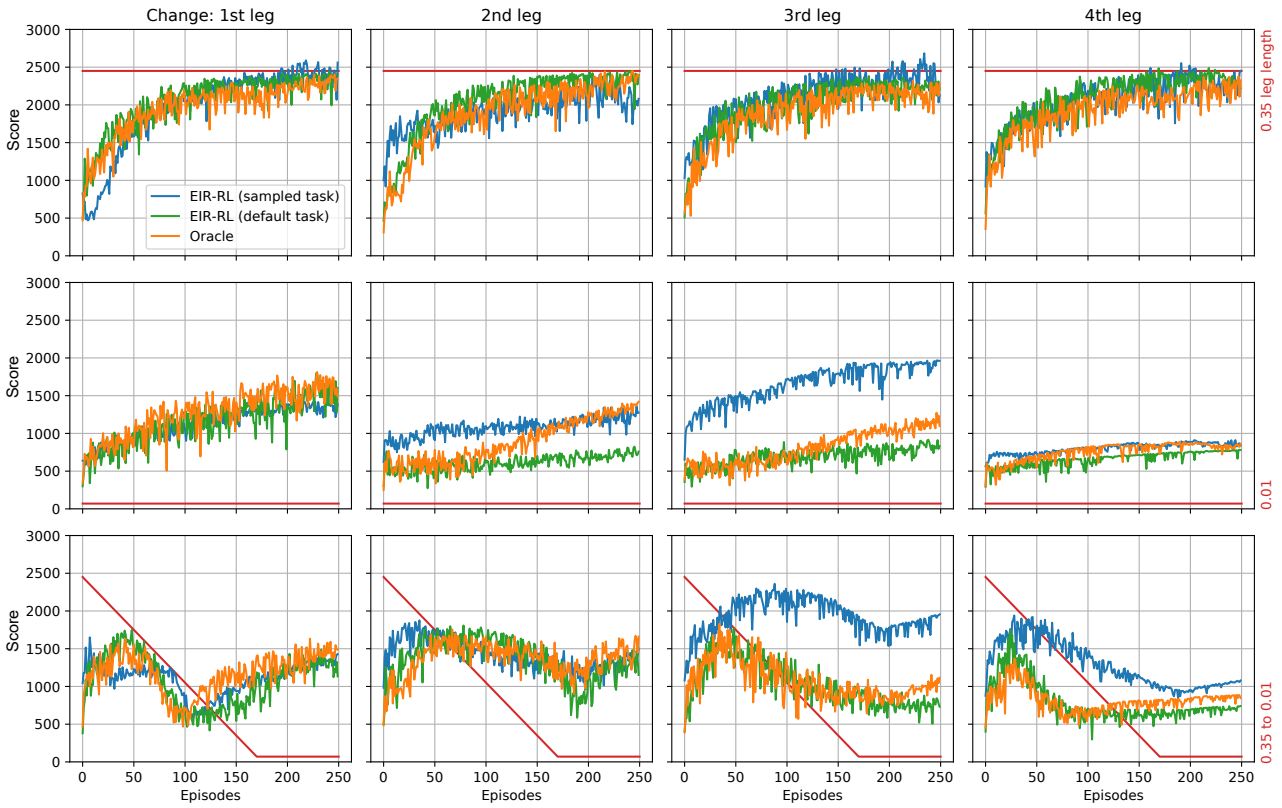

Figure 3: Ant: PPO Training Curves. Testing EIR-RL with different leg changes (indicated by the red lines) after an initial training phase under default settings. The IR network trained with changes to the third leg showed an advantage in the same setting. Otherwise, training curves are similar across the three different reward signals.

4 CONCLUSION

In this paper, we presented an approach to evolve a reward function to use for reinforcement learning to replace the reward signal provided by the environment. The reward function only relied on information that is readily available to the agent during deployment after training. The main purpose of such an evolved function is to enable continued training after deployment, such that the agent can recover performance in the case the environment changes after the training phase.

In addition to enabling sustained policy updates after an initial training phase, we observed several advantages of training with the evolved internal reward signal. These observed benefits included faster training speed in default environments and improved training stability. When adding more environmental settings in the `Ant-v3` environment during the optimization of the internal reward network, an increased recovery of performance was observed if a similar change occurred during testing. This happened without showing any negative consequences in terms of recovery in settings unseen during training. This result is particularly encouraging, as potential environmental changes can be the reason for the need for extra training in the first place. The wider the range of changes that it is possible to recover from at a faster speed, the more useful will the evolved reward function be.

Overall, the experiments presented in this paper demonstrate the importance of giving adequate rewards to reinforcement learners and that rewards tailored by evolution can make meaningful differences in the training of such learners. This is shown with a meta-learning design that is simple both conceptually and in terms amount of parameters evolved. The EIR-RL approach as presented here has the potential to be useful as is, but the simplicity of the method also makes it well-positioned to be combined with other meta-learning and reinforcement learning approaches in future studies of autonomous, adapting agents.

REFERENCES

[1] Gilbert Feng, Hongbo Zhang, et al. 2022. Genloco: Generalized locomotion controllers for quadrupedal robots. *arXiv preprint arXiv:2209.05309* (2022).
[2] Nikolaus Hansen. 2006. The CMA evolution strategy: a comparing review. *Towards a new evolutionary computation* (2006), 75–102.
[3] Louis Kirsch, James Harrison, et al. 2022. General-purpose in-context learning by meta-learning transformers. *arXiv preprint arXiv:2212.04458* (2022).
[4] Joachim Winther Pedersen and Sebastian Risi. 2021. Evolving and merging hebbian learning rules: increasing generalization by decreasing the number of rules. *arXiv preprint arXiv:2104.07959* (2021).
[5] John Schulman, Filip Wolski, et al. 2017. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347* (2017).
[6] Laura Smith, Ilya Kostrikov, and Sergey Levine. 2022. A walk in the park: Learning to walk in 20 minutes with model-free reinforcement learning. *arXiv preprint arXiv:2208.07860* (2022).
[7] Xingyou Song, Yiding Jiang, et al. 2019. Observational overfitting in reinforcement learning. *arXiv preprint arXiv:1912.02975* (2019).